



Material de Clase 06

Comenzaremos esta clase con algunos conceptos que tratarán de dejar en claro la diferencia entre los sistemas monolíticos y los sistemas orientados a servicios.

La interconexión entre sistemas o utilizar servicios que ofrecían otros sistemas desde nuestra aplicación era bastante difícil.

- Las empresas creaban sus propios sistemas con sus datos residentes en el mismo sitio.
- Eran métodos “monolíticos”, con necesidades casi nulas de integración.
- Necesitaba **una integración muy profunda, generalmente de punto a punto**

Con el tiempo surgieron sistemas especializados integrando distintos servicios que consumían las áreas de una organización. (ERP, CRMS, CMS..)

Estos sistemas eran desarrollados por diferentes empresas y tecnologías donde se encontraban problemas de integración e interoperabilidad.

Para ayudar a solucionar este problema surgieron varias tecnologías:

- **DCOM** (Distributed Component Object Model) utilizando **RCP** (Remote Procedure Call) que soporta **mensajes muy básicos**.
- **CORBA** (Common Object Request Broker Architecture) que **requiere alto acoplamiento entre el cliente y servidor**.

Luego, en 1996, hace su aparición SOA (Arquitectura orientada a servicios) • **SOA fue descrita por primera vez en el año 1996** (Gartner).

SOA Incentiva a las empresas a **crear servicios interoperables**

Solo **promueve la creación de servicios para lograr la integración de aplicaciones**, por lo que se podría decir que **CORBA es el precursor de SOA**

- Permite que los **desarrollos orientados a servicios sean posibles**.
- SOA como arquitectura no solo trae el concepto de crear servicios sino un **nuevo paradigma que revoluciono la forma en que construimos software de hoy en día**.

El modelo de programación CORBA

CORBA es una especificación para crear aplicaciones distribuidas basadas en objetos. La arquitectura y la especificación de CORBA fueron desarrolladas por Object Management Group (OMG). El OMG es un consorcio de varios cientos de proveedores de sistemas de información. El objetivo de CORBA es promover un enfoque

orientado a objetos para construir e integrar aplicaciones de software distribuidas.

La especificación CORBA proporciona un modelo amplio y coherente para crear aplicaciones distribuidas al definir:

- Un modelo de objetos para crear aplicaciones distribuidas.
- Un conjunto común de objetos de programación de aplicaciones que utilizarán las aplicaciones cliente y servidor.
- Una sintaxis para describir las interfaces de los objetos utilizados en el desarrollo de aplicaciones distribuidas.
- Soporte para el uso de aplicaciones escritas en múltiples lenguajes de programación.

La especificación CORBA describe cómo desarrollar una implementación de CORBA. También describe los enlaces del lenguaje de programación que los desarrolladores usan para desarrollar aplicaciones.

Para ilustrar las ventajas de utilizar la arquitectura CORBA, esta sección compara las primeras técnicas de desarrollo de aplicaciones cliente/servidor con las técnicas de desarrollo CORBA.

Enfoque anterior a CORBA para el desarrollo de cliente/servidor

La computación cliente/servidor es una metodología de desarrollo de aplicaciones que permite a los programadores distribuir el procesamiento entre sistemas de máquinas en red, lo que permite un uso más eficiente de los recursos de la máquina. En la informática cliente/servidor, una aplicación consta de dos partes: la aplicación cliente y la aplicación servidor. Estas dos aplicaciones normalmente se ejecutan en diferentes máquinas, conectadas por una red, como se muestra en la siguiente figura.

La aplicación cliente realiza solicitudes de información o servicios y, por lo general, proporciona a los usuarios un medio para mostrar los resultados. La aplicación del servidor cumple con las solicitudes de una o más aplicaciones cliente y, por lo general, realiza funciones de cómputo intensivo.

Las principales ventajas del modelo cliente/servidor son:

- Las funciones informáticas se ejecutan en el sistema de máquina más adecuado.
- Los desarrolladores pueden equilibrar la carga de procesamiento de aplicaciones entre varios servidores.
- Las aplicaciones de servidor se pueden compartir entre numerosas aplicaciones de cliente.

Por ejemplo, los sistemas de escritorio brindan a muchos usuarios comerciales un entorno gráfico fácil de usar para mostrar información. Sin embargo, los sistemas de escritorio pueden tener espacio en disco y memoria restringidos y, por lo general, son sistemas de un solo usuario. Los sistemas de máquinas más grandes y potentes son más adecuados para realizar funciones de cómputo intensivo y proporcionar acceso a múltiples usuarios y acceso a bases de datos compartidas.

Por lo tanto, los sistemas más grandes normalmente ejecutan la parte del servidor de la aplicación. De esta forma, los sistemas de escritorio distribuidos y los servidores en red proporcionan un entorno informático perfecto para implementar aplicaciones cliente/servidor distribuidas.

Aunque el enfoque cliente/servidor no CORBA proporciona los medios para distribuir el procesamiento en una red heterogénea, tiene las siguientes desventajas:

- Para las comunicaciones, la aplicación del cliente debe saber cómo acceder a la aplicación del servidor, incluida cualquier información de protocolo de red necesaria.

Las aplicaciones cliente/servidor pueden usar el mismo protocolo de red único o diferentes protocolos. Si utilizan múltiples protocolos, las aplicaciones deben repetir lógicamente el código específico del protocolo para cada red.

- Las aplicaciones deben manejar conversiones de formato de datos cuando se integran con máquinas que usan diferentes formatos de datos.

Por ejemplo, algunas máquinas leen un valor entero desde la dirección de byte más baja hasta la más alta (little endian), mientras que otras leen la dirección de byte más alta hasta la más baja (big endian). Algunos sistemas de máquinas también pueden usar diferentes formatos para números de punto flotante o cadenas de texto. Si una aplicación envía datos a una máquina que usa un formato de datos diferente, pero la aplicación no convierte los datos, los datos se malinterpretan.

El transporte de datos a través de la red y su conversión a su representación adecuada en el sistema de destino se denomina clasificación de datos. En muchos modelos de cliente/servidor que no son CORBA, las aplicaciones deben realizar todo el cálculo de referencias de datos. La clasificación de datos requiere que la aplicación use características de la red y el sistema operativo para mover datos de una máquina a otra. También requiere que la aplicación realice todas las traducciones de formato de datos para garantizar que los datos se lean de la misma manera que se enviaron.

- Hay menos flexibilidad para la extensión de la aplicación.

El enfoque cliente/servidor no CORBA une las aplicaciones cliente y servidor. Por lo tanto, si cambia la aplicación del cliente o del servidor, el programador debe cambiar la interfaz, la dirección de red y el transporte de red. Además, si el programador transfiere las aplicaciones de cliente y servidor a una máquina que admita una interfaz de red diferente, el programador debe crear una nueva interfaz de red para esas aplicaciones.

Enfoque CORBA para el desarrollo de cliente/servidor

El modelo CORBA proporciona un enfoque más flexible para desarrollar aplicaciones distribuidas. El modelo CORBA:

- Separa formalmente las partes de cliente y servidor de la aplicación.

Una aplicación cliente CORBA solo sabe cómo pedir que se haga algo, y una aplicación servidor CORBA solo sabe cómo realizar una tarea que una aplicación cliente le ha pedido que haga. Debido a esta separación, los desarrolladores pueden cambiar la forma en que un servidor realiza una tarea sin afectar la forma en que la aplicación cliente solicita que la aplicación del servidor realice la tarea.

- Separa lógicamente una aplicación en objetos que pueden realizar ciertas tareas, llamadas operaciones.

CORBA se basa en el modelo de computación de objetos distribuidos, que combina los conceptos de computación distribuida (cliente y servidor) y computación orientada a objetos (basada en objetos y operaciones).

En la informática orientada a objetos, los objetos son las entidades que componen la aplicación y las operaciones son las tareas que un servidor puede realizar en esos objetos. Por ejemplo, una aplicación bancaria podría tener objetos para cuentas de clientes y operaciones para depositar, retirar y ver el saldo en las cuentas.

- Proporciona clasificación de datos para enviar y recibir datos con aplicaciones de máquinas locales o remotas

Por ejemplo, el modelo CORBA formatea automáticamente para big o little endian según sea necesario. (Consulte la sección anterior para obtener una descripción de la clasificación de datos).

- Oculta las interfaces de protocolo de red de las aplicaciones.

El modelo CORBA maneja todas las interfaces de red. Las aplicaciones solo ven objetos. Las aplicaciones pueden ejecutarse en diferentes máquinas y, debido a que el ORB maneja todo el código de la interfaz de red, la aplicación no requiere ningún cambio relacionado con la red si luego se implementa en una máquina que admita un protocolo de red diferente.

El modelo CORBA permite que las aplicaciones cliente realicen solicitudes a las aplicaciones del servidor y reciban respuestas de ellas sin conocimiento directo de la fuente de información o su ubicación. En un entorno CORBA, las aplicaciones no necesitan incluir información sobre la red y el sistema operativo para comunicarse; en cambio, las aplicaciones de cliente y servidor se comunican con Object Request Broker (ORB). La siguiente figura muestra el ORB en un entorno cliente/servidor.

CORBA define el ORB como un intermediario entre las aplicaciones cliente y servidor. El ORB entrega las solicitudes de los clientes a las aplicaciones de servidor correspondientes y devuelve las respuestas del servidor a la aplicación de cliente solicitante. Usando un ORB, una aplicación de cliente puede solicitar un servicio sin conocer la ubicación de la aplicación de servidor o cómo la aplicación de servidor cumplirá con la solicitud.

En el modelo CORBA, las aplicaciones cliente solo necesitan saber qué solicitudes pueden realizar y cómo realizarlas; no necesitan estar codificados con ningún detalle de implementación del servidor o de los formatos de datos. Las aplicaciones de servidor solo necesitan saber cómo cumplir con las solicitudes, no cómo devolver datos a la aplicación cliente.

Esto significa que los programadores pueden cambiar la forma en que una aplicación de servidor realiza una tarea sin afectar la forma en que la aplicación cliente solicita que la aplicación de servidor realice esa tarea. Por ejemplo, mientras las interfaces entre las aplicaciones cliente y servidor no cambien, los programadores pueden evolucionar y crear nuevas implementaciones de una aplicación servidor sin cambiar la aplicación cliente; además, pueden crear nuevas aplicaciones cliente sin cambiar las aplicaciones del servidor.

COMPONENTES DE CORBA.

Como se explicó anteriormente, es un estándar para distribuir objetos a través de redes distribuidas.

No está asociado con ningún lenguaje en particular pero puede utilizarse con cualquiera que pueda implementar objetos CORBA. Los objetos se describen en una interface de definición de lenguajes (IDL).

IDL.

Para identificar los servicios que los objetos ofrecen se necesita de un lenguaje independiente y bien definido. La norma ISO/IEC 14750 (ITU T X.920) define dicho lenguaje, al que se conoce como lenguaje de definición de interfaces de ODP, o ODP IDL por su acrónimo en inglés.

En IDL, una interfaz es una descripción de un conjunto de posibles operaciones que un cliente puede solicitar de un objeto. El objeto satisface una interfaz si este puede satisfacer una solicitud de otro objeto. La interface provee mecanismos compuestos que le permiten al objeto soportar varias interfaces.

Repositorio de interfaces.

El repositorio de interfaces (IR) es un servicio que ofrece objetos persistentes que representan la información IDL de las interfaces disponibles en CORBA, de una forma accesible en tiempo de ejecución. Esta información puede ser utilizada por el ORB para realizar peticiones.

La interfaz de invocación dinámica El DII (Dynamic Invocation Interface) es una interfaz que nos permite la construcción dinámica de invocaciones para un determinado objeto.

La invocación es idéntica a la que llega a través de la interfaz estática pero que ya dentro del cliente, logra una flexibilidad en arquitecturas complejas y dinámicas. Una invocación dinámica se compone, de una referencia al objeto, una operación y una lista de parámetros. Todos estos datos se obtienen del Repositorio de Interfaces (IR).

STUB.

Es el intermediario entre el cliente y el ORB. Toma las llamadas de los clientes y las entrega al ORB. Se requiere una clase de stub por cada clase remota. El stub permite que el desarrollador no tenga que trabajar con instrucciones remotas porque son objetos que residen del lado del cliente y representan a los objetos remotos. Aquí se indentifica: Host, puerto e identificador del objeto.

CORBA incluye **cuatro componentes**:

Skeleton.

Es el intermediario entre ORB y los objetos del servidor . Recibe llamadas del ORB y ejecuta los métodos correspondientes en el servidor sobre el objeto llamado. Cuando el cliente establece un objeto local (con servicio remoto), la petición se realiza por intermedio del protocolo de comunicaciones IIOP a través del ORB. El servidor recibe la petición, busca el objeto definido (compara el esqueleto del método en el módulo esqueleto) lo ejecuta y retorna la respuesta al cliente

ORB (Object Request Broker)

ORB es el componente que gestiona la comunicación de parámetros. De esta manera el manejo es transparente para el cliente que invoca el servicio y el servidor que lo ofrece y soporta cuatro tipos de interfaces de objetos:

- **Object Services:** Son interfaces para servicios generales. Son usadas en cualquier programa basado en objetos distribuidos.
- **Common Facilities:** Son interfaces orientadas al usuario final y que se programan por la aplicación específica.
- **Domain Interfaces:** Son interfaces de dominio específico para las aplicaciones.
- **Application Interfaces:** Este tipo de interfaz acepta interfaces que no sean estandarizadas y se utilizan en aplicaciones específicas.

Servidor CORBA

El servidor CORBA crea objetos CORBA y los inicializa con un ORB. El servidor coloca las referencias a los objetos CORBA dentro de un servicio de denominación de modo que los clientes puedan acceder a los mismos.

Servicio de nombres

El servicio es el que mantiene la referencias a los objetos CORBA.

Nodo CORBA Request

Es el que actúa como cliente CORBA.



Recurso de información

Para una mejor comprensión de CORBA y sistemas distribuidos sugerimos leer:

Sistemas distribuidos de Santiago Sabato capítulo 2 y 4



Bibliografía y recursos

El **material** de la clase lo encontramos en las siguientes diapositivas que preparamos:



SISTEMAS DISTRIBUIDOS.pdf

